Predator–Prey Reinforcement-Learning Ecosystem

Mathematics, algorithms and illustrations of the from-scratch implementation

Project documentation (Python + NumPy, no ML frameworks)

November 5, 2026

Massine TALBI

Abstract

This document lists every equation used by the code: world kinematics, stamina, observations, the hand-written neural network and Adam optimiser, the Gaussian actor–critic with PPO-clip and GAE, and the shark reward with all coefficients explained. The fish learning system (Phases 7–9) is specified at the end but is *not yet implemented*; currently fish are random wanderers and only the sharks learn.

Contents

1 Software architecture **2**

2 World kinematics **2**
2.1 Position, heading, speed . . . 2
2.2 Fish: Ornstein–Uhlenbeck wandering . . . 2
2.3 Sharks: controlled by the network . . . 3
2.4 Walls and collisions . . . 3

3 Shark stamina **3**

4 Observations **3**

5 The neural network (hand-written) **4**
5.1 Forward pass and initialisation . . . 4
5.2 Back-propagation . . . 4
5.3 Adam optimiser and gradient clipping . . . 5

6 Reinforcement-learning algorithm: PPO actor–critic **5**
6.1 Why this algorithm . . . 5
6.2 Gaussian policy . . . 5
6.3 Returns and advantages (GAE) . . . 5
6.4 PPO-clip objective and manual gradients . . . 5

7 Shark reward (“points”) **6**

8 Training loop **7**

9 Results of the current implementation **7**

10 Planned: fish reward (Phases 7–9, not yet implemented) **7**

11 Verification summary **8**

1 Software architecture

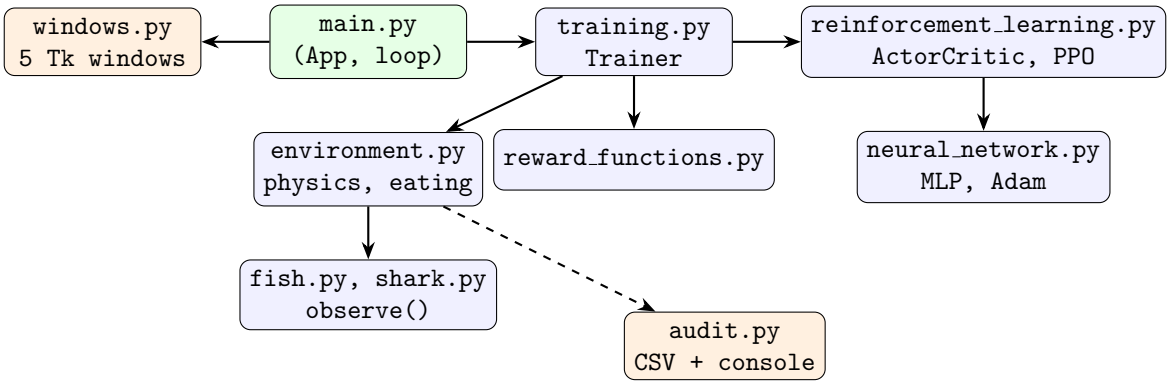


Figure 1: Module responsibilities. The Tk windows read state from the `App`; they never modify the physics directly.

2 World kinematics

Physics runs with a fixed step $\Delta t = 1/60\,\mathrm{s}$; the speed controller only changes how many steps are executed per frame, so results are identical at any speed.

2.1 Position, heading, speed

For every agent

$$\theta_{t+1} = (\theta_t + \omega_t \Delta t) \bmod 2\pi, \tag{1}$$

$$x_{t+1} = x_t + v_t \cos\theta_{t+1}\,\Delta t, \qquad y_{t+1} = y_t + v_t \sin\theta_{t+1}\,\Delta t. \tag{2}$$

2.2 Fish: Ornstein–Uhlenbeck wandering

Fish have *no intelligence*; their randomness is smooth. Turn rate ω and speed v follow

$$\omega_{t+1} = \omega_t - \kappa\,\omega_t \Delta t + \sigma_\omega \sqrt{\Delta t}\,\xi_t, \quad \xi_t \sim \mathcal{N}(0,1), \tag{3}$$

$$v_{t+1} = v_t + \kappa_v (v_{\text{cruise}} - v_t)\Delta t + \sigma_v \sqrt{\Delta t}\,\xi'_t. \tag{4}$$

The stationary standard deviation of ω is $\sigma_\omega/\sqrt{2\kappa} = 2.5/\sqrt{3} \approx 1.44\,\mathrm{rad/s}$ (clipped at ± 3, measured 1.34).

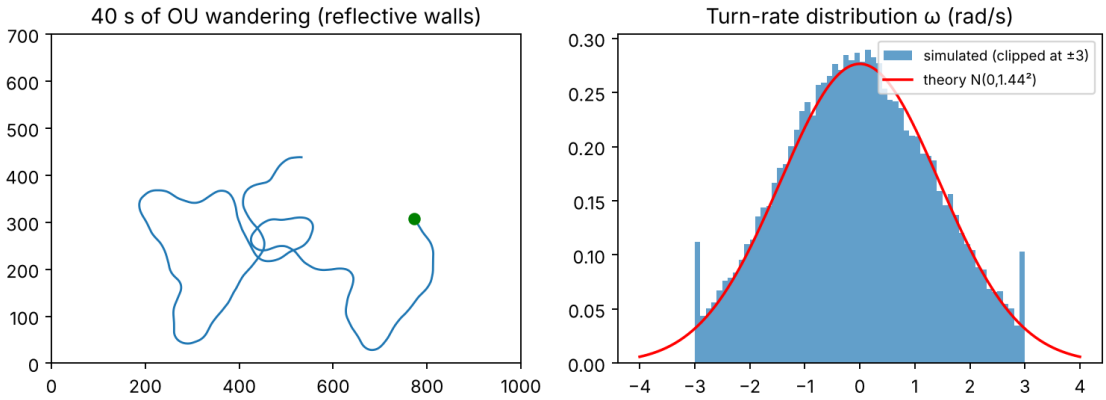


Figure 2: Left: a simulated fish path. Right: measured turn-rate distribution vs. the OU theory.

2.3 Sharks: controlled by the network

The network outputs two commands $u_\omega, u_v \in [-1, 1]$ (clipped):

$$\omega = \omega_{\max} u_\omega, \qquad \dot{v} = k_a\big(v_{\text{tgt}} - v\big), \quad v_{\text{tgt}} = v_{\max}\tfrac{u_v+1}{2}, \; k_a = 4\,\text{s}^{-1}. \tag{5}$$

2.4 Walls and collisions

Reflective walls: hitting a vertical wall maps $\theta \mapsto \pi - \theta$, a horizontal wall $\theta \mapsto -\theta$ (the agent is clamped inside by its radius). A shark at $\mathbf{p}_s$ catches a fish at $\mathbf{p}_f$ when

$$\|\mathbf{p}_s - \mathbf{p}_f\|^2 < (R_{\text{att}} + r_f)^2, \tag{6}$$

evaluated for all pairs with one vectorised distance matrix. At most one fish per shark per step; the eaten fish respawns at a random place, so prey density stays constant.

3 Shark stamina

Let $E \in [0, E_{\max}]$ be stamina, $s = v/v_{\max}$. Per second

$$\dot{E} = -\big(e_0 + e_1 s^2\big) + \rho\mathbb{1}[s < s_{\text{rest}}], \qquad E \leftarrow \min(E_{\max}, E + G) \text{ on eating}, \tag{7}$$

with $e_0 = 1$, $e_1 = 4$, $\rho = 2$, $s_{\text{rest}} = 0.25$, $G = 25$ (adjustable live). The quadratic term makes fast swimming disproportionately expensive (as drag does), so resting and cruising are genuinely cheaper; the shark dies when $E \le 0$.

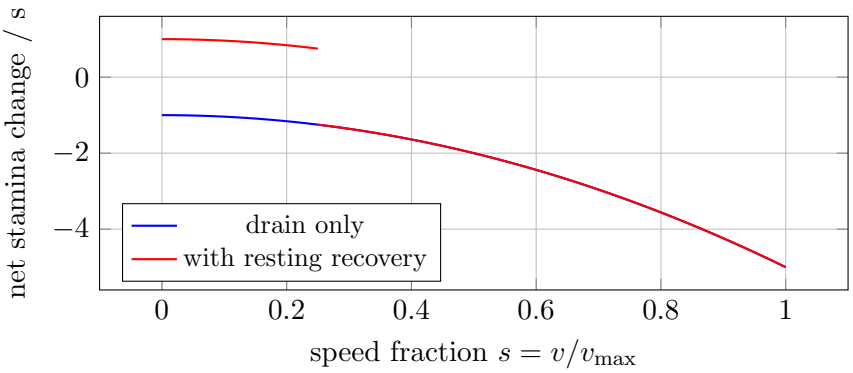


Figure 3: Stamina rate. Below $s = 0.25$ the shark gains $+\rho$; above it only drains.

4 Observations

Each shark sees the world in its own frame ($+x$ along its heading). For the $k = 3$ nearest fish with world offset $\mathbf{d} = (\Delta x, \Delta y)$ and relative velocity $\Delta\mathbf{v} = \mathbf{v}_f - \mathbf{v}_s$:

$$\begin{pmatrix} o_x \\ o_y \end{pmatrix} = \frac{1}{S_p}\begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix}\begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}, \qquad \begin{pmatrix} o_{vx} \\ o_{vy} \end{pmatrix} = \frac{1}{S_v} R(-\theta)\Delta\mathbf{v}, \tag{8}$$

clipped to $[-1, 1]$ ($S_p = 300\,\text{px}$, $S_v = 150\,\text{px/s}$). Three more inputs: fish density $n_{<150\text{px}}/10$ (clipped to 1), stamina $E/E_{\max}$, own speed $v/v_{\max}$. Total: $\dim \mathbf{s} = 3 \times 4 + 3 = 15$.

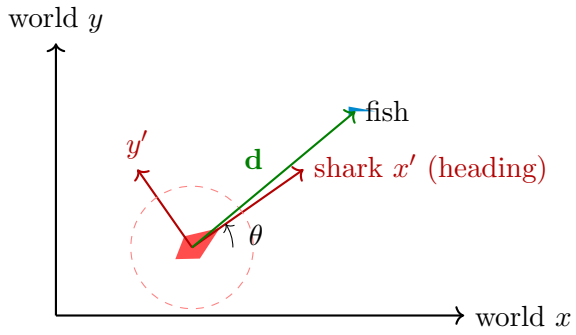


Figure 4: Egocentric frame: $\mathbf{d}$ is rotated by $-\theta$ so the network is invariant to absolute orientation.

5 The neural network (hand-written)

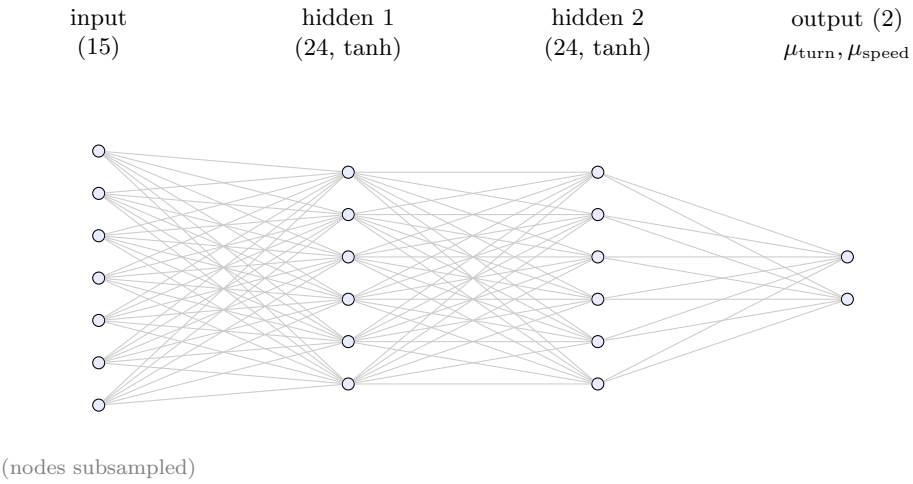


Figure 5: Actor $15 \to 24 \to 24 \to 2$. The critic is a separate $15 \to 24 \to 1$ network. The GUI window "Shark Neural Network" colours nodes by activation and edges by the signal $w_{ij}a_i$.

5.1 Forward pass and initialisation

For layer l with input $\mathbf{a}^{l-1}$:

$$\mathbf{z}^l = \mathbf{a}^{l-1}W^l + \mathbf{b}^l, \qquad \mathbf{a}^l = \tanh(\mathbf{z}^l) \text{ (hidden)}, \qquad \mathbf{a}^L = \mathbf{z}^L \text{ (output, linear)}. \tag{9}$$

Weights use Xavier/Glorot uniform initialisation, $W_{ij} \sim U(-\ell, \ell)$, $\ell = \sqrt{6/(n_{in} + n_{out})}$, which keeps activation variance roughly constant through tanh layers; biases start at 0. The actor's last layer is multiplied by 0.01 so the initial policy is $\mu \approx 0$: pure noise, i.e. random behaviour.

5.2 Back-propagation

With $\delta^L = \partial\mathcal{L}/\partial\mathbf{a}^L$ (computed by the RL loss) and $\tanh'(z) = 1 - a^2$:

$$\frac{\partial\mathcal{L}}{\partial W^l} = (\mathbf{a}^{l-1})^\top \delta^l, \qquad \frac{\partial\mathcal{L}}{\partial \mathbf{b}^l} = \textstyle\sum_{\text{batch}} \delta^l, \tag{10}$$

$$\delta^{l-1} = \left(\delta^l\,(W^l)^\top\right) \odot \left(1 - (\mathbf{a}^{l-1})^{\odot 2}\right). \tag{11}$$

Verified in `tests/test_all.py` against central finite differences $\frac{\mathcal{L}(w+h)-\mathcal{L}(w-h)}{2h}$, $h = 10^{-6}$ (agreement $< 10^{-5}$).

5.3 Adam optimiser and gradient clipping

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{12}$$

$$\theta_t = \theta_{t-1} - \eta \, \frac{m_t/(1-\beta_1^t)}{\sqrt{v_t/(1-\beta_2^t)} + \varepsilon}, \tag{13}$$

$\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$; $\eta_{\text{actor}} = 10^{-3}$, $\eta_{\text{critic}} = 3 \cdot 10^{-3}$. Gradients are rescaled when $\|g\|_2 > 0.5$.

6 Reinforcement-learning algorithm: PPO actor–critic

6.1 Why this algorithm

Actions are continuous (turn, speed), so value-based methods such as DQN would need a coarse discretisation that wrecks smooth interception. A stochastic Gaussian policy gives exploration for free and its log-probability is differentiable. Plain REINFORCE has very high variance, so we add a critic baseline (actor–critic) and Generalised Advantage Estimation. PPO's clipped objective stops one batch from pushing the policy too far, which matters in a non-stationary multi-agent world. All sharks share *one* policy (parameter sharing): N sharks produce $N\times$ the experience for the same network.

6.2 Gaussian policy

$$\pi_\theta(\mathbf{a}|\mathbf{s}) = \mathcal{N}\big(\mu_\theta(\mathbf{s}), \operatorname{diag}(\sigma^2)\big), \quad \sigma_i = e^{\lambda_i} \ (\lambda \text{ learnable log-std}), \tag{14}$$

$$\log \pi = -\tfrac{1}{2}\sum_i z_i^2 - \sum_i \lambda_i - \tfrac{k}{2}\log 2\pi, \quad z_i = \frac{a_i - \mu_i}{\sigma_i}, \tag{15}$$

$$\frac{\partial \log \pi}{\partial \mu_i} = \frac{z_i}{\sigma_i}, \qquad \frac{\partial \log \pi}{\partial \lambda_i} = z_i^2 - 1, \qquad H = \sum_i \Big(\lambda_i + \tfrac{1}{2}\log(2\pi e)\Big). \tag{16}$$

Exploration vs. exploitation is therefore controlled by σ, which the optimiser shrinks as the policy becomes confident (clamped to $\lambda \in [-2, 0.5]$; entropy bonus $c_{ent} = 10^{-3}$ slows premature collapse).

6.3 Returns and advantages (GAE)

With discount $\gamma = 0.99$ per 0.1 s decision (horizon ≈ 10 s) and $\lambda_{GAE} = 0.95$:

$$\delta_t = r_t + \gamma(1-d_t)V(\mathbf{s}_{t+1}) - V(\mathbf{s}_t), \qquad \hat{A}_t = \sum_{l \ge 0} (\gamma\lambda_{GAE})^l \, \delta_{t+l} \ \text{(stopped at episode ends } d_t = 1), \tag{17}$$

$$\hat{R}_t = \hat{A}_t + V(\mathbf{s}_t). \tag{18}$$

Advantages are normalised per batch: $\hat{A} \leftarrow (\hat{A} - \bar{A})/(\operatorname{std}(\hat{A}) + 10^{-8})$.

6.4 PPO-clip objective and manual gradients

Let $\rho = \exp(\log \pi_\theta - \log \pi_{old})$ and $\epsilon = 0.2$:

$$L^{CLIP} = \mathbb{E}\Big[\min\big(\rho\hat{A},\ \operatorname{clip}(\rho, 1-\epsilon, 1+\epsilon)\hat{A}\big)\Big], \tag{19}$$

$$\mathcal{L}_\pi = -L^{CLIP} - c_{ent} H, \qquad \mathcal{L}_V = \tfrac{1}{2}\mathbb{E}\big[(V(\mathbf{s}) - \hat{R})^2\big]. \tag{20}$$

The gradient w.r.t. $\log \pi$ is non-zero only where the clip is inactive:

$$\frac{\partial \mathcal{L}_\pi}{\partial \log \pi_b} = -\frac{\rho_b \hat{A}_b}{B} \mathbb{1}_{\text{active}}, \quad \mathbb{1}_{\text{active}} = \neg\big[(\hat{A} > 0 \wedge \rho > 1+\epsilon) \vee (\hat{A} < 0 \wedge \rho < 1-\epsilon)\big]. \tag{21}$$

Chain rule with the Gaussian derivatives above gives $\partial \mathcal{L}/\partial \mu_{b,i} = g_b z_{b,i}/\sigma_i$ and $\partial \mathcal{L}/\partial \lambda_i = \sum_b g_b(z_{b,i}^2 - 1) - c_{ent}$ with $g_b = \partial \mathcal{L}_\pi / \partial \log \pi_b$; $\partial \mu / \partial \theta$ comes from back-propagation. The critic gradient is $c_V(V - \hat{R})/B$ with $c_V = 0.5$. Each batch (1024 transitions) is used for 4 epochs of mini-batches of 256. This actor gradient is also verified by finite differences in the test suite.

7 Shark reward ("points")

Each decision (0.1 s) a shark receives $r = r_{\text{eat}} + r_{\text{energy}} + r_{\text{time}} + r_{\text{shape}} + r_{\text{low}} + r_{\text{death}}$, and the GUI shows the running total of every component:

$$r_{\text{eat}} = R\, n_{\text{eaten}}, \qquad r_{\text{energy}} = -\lambda_E\, \Delta E_{\text{spent}}, \tag{22}$$

$$r_{\text{time}} = -\lambda_t\, \Delta t_{dec}, \qquad r_{\text{low}} = -\lambda_{low}\big[\max(0,\ \phi_{low} - E/E_{\max})\big]^2, \tag{23}$$

$$r_{\text{shape}} = \lambda_p\big(\gamma\, \Phi(\mathbf{s}') - \Phi(\mathbf{s})\big), \qquad r_{\text{death}} = -R_d \mathbb{1}[E \le 0], \tag{24}$$

with potential $\Phi(\mathbf{s}) = -\min(d_{\text{nearest}}, d_{\max})/d_{\max}$ ($\Phi = 0$ right after a catch).

Symbol	Value	Why this value
R	10	Dominant signal: a catch must outweigh anything accumulated while hunting.
λ_E	0.05	Full-speed costs 5 units/s $\Rightarrow$ -0.025 per decision; cruising ≈ -0.01. Small but systematically favours efficient motion ("wasting stamina is negative").
λ_t	0.1 /s	-0.01 per decision: mild urgency; a shark that never catches loses points.
$\lambda_p, d_{\max}$	1.0, 400 px	Dense hint for interception. Potential-based (Ng et al. 1999), so it *cannot* change the optimal policy; it only speeds credit assignment.
$\lambda_{low}, \phi_{low}$	2.0, 0.3	Quadratic penalty (max -0.18/step at $E = 0$) teaches "keep stamina in reserve".
R_d	5	Starvation is clearly worse than a missed chase, but smaller than R so sharks still dare to hunt.
γ	0.99	Credit horizon ≈ 10 s, long enough to link a chase to its catch.

Table 1: Shark reward coefficients.

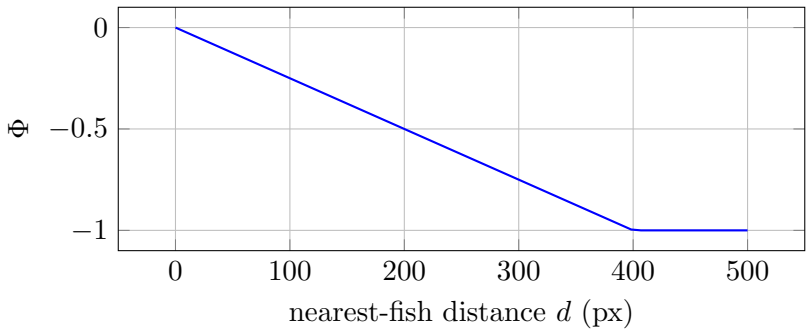


Figure 6: Shaping potential: closing the distance yields $\gamma\Phi(s') - \Phi(s) > 0$.

8 Training loop

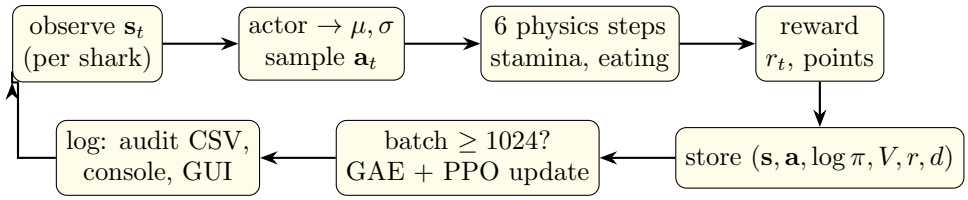


Figure 7: One decision interval. A dead shark respawns with full stamina and the same shared policy; shark count can change at any time.

9 Results of the current implementation

Three sharks, 50 fish, ∼10 000 simulated seconds (292 PPO updates, 3 minutes of CPU):

PPO update	1 (random)	50	292
fish eaten / shark / min	15.2	45.0	77.2
mean step reward	+0.24	+0.73	+1.27



Figure 8: Learning curves (thin: raw, thick: 10-update moving average).

Honest caveat. With 50 fish in a 1000 × 700 world even random sharks eat often, so no shark starved in this run; stamina management becomes relevant when you lower the fish count or the stamina gain per fish (both adjustable in the Controls window). The result shows learned pursuit/interception (a 5× gain), not yet a full ecological strategy.

10 Planned: fish reward (Phases 7–9, not yet implemented)

With d_s the distance to the nearest shark, $g \in [0,1]$ a group-cohesion score (fraction of neighbours within the comfort radius), $a \in [0,1]$ heading alignment with neighbours, E_f fish stamina and ι an isolation indicator:

$$r_f = \underbrace{\alpha_s}_{\text{survive}} + \alpha_d \,\mathrm{sat}\!\left(\frac{d_s - r_{\text{danger}}}{r_{\text{safe}}}\right) + \alpha_c(1+\eta g)\,\mathbb{1}_{\text{evaded}} + \alpha_g\, g + \alpha_a\, a - \alpha_e\,[\,\phi - E_f]_+^2 - \alpha_\iota\,\iota - \alpha_x\,\mathbb{1}_{\text{eaten}}. \quad (25)$$

The cooperative bonus is $\alpha_c(1+\eta g)$ rather than $\alpha_c\, g$: a lone fish that escapes still earns $\alpha_c > 0$ (individual survival is never penalised), while a fish escaping inside the school earns up to $(1+\eta)$ times more. Coefficients will be derived and tested phase by phase, exactly like the shark's.

11 Verification summary

- MLP back-propagation and the PPO actor gradient: finite-difference checks.
- GAE on a hand-computed two-step case ($\gamma = 0.5, \lambda = 1$ gives $\hat{A} = [1.5, 1.0]$).
- Reward signs (catch $>$ idle $>$ starvation; approaching $>$ retreating).
- Walls, reproducibility by seed, eating adds stamina, death at zero stamina, live add/remove of sharks, finite weights after training.